[46] N. Belrose, D. Schneider-Joseph, S. Ravfogel, R. Cotterell, E. Raff, and S. Biderman, "LEACE: Perfect linear concept erasure in closed form," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[47] J. Feng, S. Russell, and J. Steinhardt, "Monitoring latent world states in language models with propositional probes," in *The Thirteenth International Conference on Learning Representations*, 2025.

[48] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *The Twelfth International Conference on Learning Representations*, 2024.

[49] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[50] H. Puerto, T. Chubakov, X. Zhu, H. T. Madabushi, and I. Gurevych, "Fine-tuning with divergent chains of thought boosts reasoning through self-correction in language models," 2024.

[51] E. Zelikman, Y. Wu, J. Mu, and N. Goodman, "STar: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems* (A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, eds.), 2022.

[52] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *The Twelfth International Conference on Learning Representations*, 2023.

[53] D. Li, S. Cao, T. Griggs, S. Liu, X. Mo, E. Tang, S. Hegde, K. Hakhamaneshi, S. G. Patil, M. Zaharia, *et al.*, "Llms can easily learn to reason from demonstrations structure, not content, is what matters!," *arXiv preprint arXiv:2502.07374*, 2025.

[54] J. Yi and J. Wang, "Shorterbetter: Guiding reasoning models to find optimal inference length for efficient reasoning," *arXiv preprint arXiv:2504.21370*, 2025.

[55] Q. Chen, L. Qin, J. WANG, J. Zhou, and W. Che, "Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[56] K. Team, A. Du, B. Gao, B. Xing, C. Jiang, C. Chen, C. Li, C. Xiao, C. Du, C. Liao, *et al.*, "Kimi k1. 5: Scaling reinforcement learning with llms," *arXiv preprint arXiv:2501.12599*, 2025.

[57] D. J. Shah, P. Rushton, S. Singla, M. Parmar, K. Smith, Y. Vanjani, A. Vaswani, A. Chaluvaraju, A. Hojel, A. Ma, *et al.*, "Rethinking reflection in pre-training," *arXiv preprint arXiv:2504.04022*, 2025.

[58] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.

[59] Y. Wang, Q. Yang, Z. Zeng, L. Ren, L. Liu, B. Peng, H. Cheng, X. He, K. Wang, J. Gao, *et al.*, "Reinforcement learning for reasoning in large language models with one training example," *arXiv preprint arXiv:2504.20571*, 2025.

[60] Z. Liu, C. Chen, W. Li, T. Pang, C. Du, and M. Lin, "There may not be aha moment in r1-zero-like training — a pilot study." `https://oatllm.notion.site/oat-zero`, 2025. Notion Blog.

# A UMAP Visualization of Self-Reflection States

To investigate how self-reflective states are internally represented, we perform UMAP-based dimensionality reduction on hidden states extracted from both Qwen2.5 1.5B and DeepSeek-R1-Distill-Qwen-1.5B models. Figure 3 in the main text illustrates the final-layer representations, while Figures 8 and 9 offer a representative view of 14 out of 28 layers across the Qwen2.5-1.5B and DeepSeek-R1-Distill-Qwen-1.5B models, respectively.

We observe that the separation between reflective and nonself-reflective hidden states is not binary, but rather continuous. As we move from shallow to deeper layers, the distinction becomes more pronounced. Early layers show considerable overlap between reflective and non-reflective tokens. In middle layers, clustering begins to emerge, and by the final layers, the separation becomes clear and robust. This progression implies that self-reflective encoding is a hierarchical feature—gradually constructed through successive transformations, rather than being localized to a single depth.

Notably, even in Qwen2.5, which rarely emits explicit self-reflection tokens, the hidden states exhibit consistent separability. This finding reinforces our central hypothesis: the capacity for self-reflection is not merely a byproduct of reinforcement learning or specialized fine-tuning (e.g., RLVR), but an inherent representational feature developed during pretraining. The existence of such a neural signature across models suggests that self-reflection occupies a distinct and manipulable subspace within the activation manifold.

These visualizations complement our quantitative results and intervention-based analyses, offering geometric intuition for the effectiveness of our vector steering method. Rather than introducing new behaviors from scratch, our method leverages existing latent structures, enhancing or suppressing reflection by operating within naturally emergent manifolds in the model's internal state space.

# B Identifying Self-Reflection Instances

To systematically identify self-reflection in language model outputs, we developed a keyword-based detection approach. We define a self-reflection instance as any generation containing explicit self-reflection tokens that signal the model's reconsideration or revision of its reasoning process.

We construct a curated list of self-reflection keywords, informed by prior analyses of reasoning dynamics in language models [7,19]. A generation is marked as self-reflective if it contains one or more of the following terms:

**Self-Reflection Keywords:**

`wait`, `re-check`, `recheck`, `check again`, `rethink`, `re-think`, `reconsider`, `re-consider`, `try again`, `re-examine`, `reexamine`, `re-evaluate`, `reevaluate`, `think again`, `consider again`, `evaluate again`, `examine again`

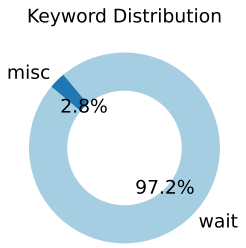


Figure 6: **Left:** List of self-reflection keywords used in our analysis. **Right:** Frequency distribution of these keywords across model-generated outputs.

To validate this detection method, we applied it to model outputs on the MATH500 benchmark using the DeepSeek-R1-Distill-Qwen-1.5B model. We set the maximum response length to 32,784 tokens to accommodate complex, multi-step solutions and to ensure that instances of late-stage self-reflection were not truncated. Among the reflection markers, the keyword `"wait"` emerged as particularly salient. In our analysis of DeepSeek-R1 outputs, `"wait"` accounted for approximately 97.2% of all detected reflection instances. This high frequency makes it a reliable and informative indicator for tracking self-reflective behavior, especially in models fine-tuned with RLVR techniques.

# C Ablation Study

We determine the optimal injection strategy via a two-stage procedure:

(i) **Scaling Search:** For each candidate layer $\ell$, we perform a grid search over $\alpha \in [-1.0, 1.0]$ to identify the value that maximizes validation performance, exploring both enhancement and suppression regimes.

(ii) **Layer Selection:** We evaluate each layer's receptivity to injection. For specialized reasoning models (e.g., DeepSeek-R1), a single well-chosen layer often suffices to yield significant gains. In contrast, for general pretrained models (e.g., Qwen2.5), we observe that distributing moderate injections across multiple layers produces the best trade-off between accuracy and efficiency.

**Effect of Injection Layer.** We've already described $\alpha$ selection in the main text. Here, we fix $\alpha = 0.01$ and examine the effect of injecting the self-reflection vector at each layer of DeepSeek-R1 1.5B on MATH-500 (Figure 7). We observe that middle layers, most notably layer 14, achieve the highest performance. Injections into early layers yield only marginal gains, as the steering signal is progressively transformed and attenuated by subsequent network operations. Conversely, injecting too late often degrades performance, likely because the intervention interferes directly with token generation rather than shaping deeper reasonin prg dynamics. These results indicate that moderate, mid-network interventions best modulate self-reflection by targeting layers that both abstract reasoning patterns and retain strong control over final predictions.

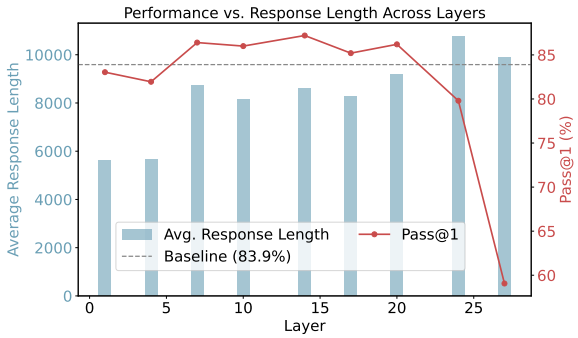


Figure 7: Effect of injecting the self-reflection vector at different layers of DeepSeek-R1 1.5B ($\alpha = 0.01$) on Pass@1 and response length for MATH-500.

# D Implementation Details

For the DeepSeek-R1-Distill-Qwen models, we adopted a specialized prompting strategy that incorporates the explicit token `<think>` to elicit self-reflective reasoning and promote internal deliberation.

In contrast, for the Qwen models, we employed the same prompt template but omitted the `<think>` token. This design allowed us to isolate and assess the specific influence of `<think>` on eliciting reflective behaviors and its downstream impact, as Qwen models do not natively rely on such explicit triggers.

To support complex multi-step reasoning, we set the maximum generation length to 32,784 tokens across all experiments, ensuring that outputs were not prematurely truncated. All experiments were conducted on a computing cluster equipped with 8 NVIDIA A5000 GPUs.

# E Limitations.

Our work presents several limitations. First, users must predefine whether to enhance or suppress self-reflection prior to inference; the model does not yet autonomously adjust its reflective behavior based on task complexity or reasoning demands. Second, our approach relies on access to internal model activations, which may not be feasible in closed-source or API-limited environments.

In the future, these limitations could be addressed by developing adaptive self-reflection mechanisms that dynamically modulate introspection based on task complexity and reasoning signals. Further research might extend these techniques to more opaque model environments with limited activation access. Additional work could also explore methods for automatic calibration of injection parameters across diverse model architectures and reasoning domains.